

Reproducible ANN Testing Framework

SIFT HDF5 Baseline on AMD Machine
(FAISS + Milvus Lite)

Konstantin Kolchin

June 24, 2026

Abstract

This document records a reproducible baseline workflow for ANN testing on an AMD machine, using ready-to-use vectors (SIFT HDF5) instead of an embedding stage. It includes setup steps, scripts to run, and measured results for FAISS and Milvus Lite. The goal is to provide a repeatable harness that any engineer can run before moving to Milvus GPU porting work.

Contents

1	Scope and Goal	2
2	Environment	2
2.1	Python packages	2
3	Dataset Acquisition	2
4	Scripts and Execution	2
4.1	FAISS baseline script	2
4.2	Milvus Lite baseline script	3
5	Measured Results	3
5.1	FAISS results	3
5.2	Milvus Lite results	3
6	Observations	3
7	Re-run Checklist	4
8	Next Steps	4
A	Teammate Run Template	4
B	Script Source: run_faiss_hdf5.py	5
C	Script Source: run_milvus_hdf5.py	6

1 Scope and Goal

The workflow validates the first part of the demo pipeline with **ready vectors** and avoids text chunking/embedding complexity in early iterations. The concrete objectives are:

- Verify dataset ingest and search benchmarking on AMD host.
- Establish reference metrics: QPS, recall@ k , p99 latency.
- Keep execution fully reproducible through explicit scripts and commands.

2 Environment

- Host: AMD machine (tested with Radeon RX 6800 XT / gfx1030).
- Working directory: `~/ann-demo`.
- Python virtual environment: `.venv`.

2.1 Python packages

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install numpy faiss-cpu h5py
pip install "pymilvus[milvus_lite]"
```

3 Dataset Acquisition

FTP source (`ftp.irisa.fr`) may hang on restricted networks. The reliable path in this run was direct HTTPS with proxy bypass:

```
cd ~/ann-demo/data
wget --no-proxy https://ann-benchmarks.com/sift-128-euclidean.hdf5
```

Dataset content check:

```
['distances', 'neighbors', 'test', 'train']
distances (10000, 100) float32
neighbors (10000, 100) int32
test      (10000, 128) float32
train     (1000000, 128) float32
```

4 Scripts and Execution

4.1 FAISS baseline script

Run:

```
python run_faiss_hdf5.py
```

The script loads `train/test/neighbors` from HDF5 and runs:

- exact baseline: `IndexFlatL2`,
- ANN baseline: `IndexIVFFlat` with variable `nprobe`.

4.2 Milvus Lite baseline script

Run:

```
python run_milvus_hdf5.py
```

Important implementation note: insertion is chunked to avoid gRPC message limit (`RESOURCE_EXHAUSTED` at 256 MB per message).

Recommended insert pattern:

```
BATCH = 50000
for s in range(0, xb.shape[0], BATCH):
    e = min(s + BATCH, xb.shape[0])
    col.insert([ids[s:e].tolist(), xb[s:e].tolist()])
col.flush()
```

5 Measured Results

5.1 FAISS results

Table 1: FAISS SIFT1M baseline ($k=10$)

Configuration	QPS	recall@10
FlatL2 (exact)	1054.5	1.0000
IVF, nprobe=1	76649.6	0.3707
IVF, nprobe=4	20394.5	0.6984
IVF, nprobe=8	10419.9	0.8336
IVF, nprobe=16	4995.9	0.9274
IVF, nprobe=32	2785.5	0.9761

5.2 Milvus Lite results

- Insert time: 79.37 s
- Index build time (IVF_FLAT): 8.80 s

Table 2: Milvus Lite IVF_FLAT results ($k=10$)

nprobe	QPS	p99 ms	recall@10
1	1490.1	475.36	0.8943
4	1465.7	465.44	0.8943
8	1518.8	456.05	0.8943
16	1474.3	466.86	0.8943
32	1535.6	473.83	0.8943

6 Observations

- FAISS behaves as expected: recall increases with `nprobe` while QPS decreases.
- Milvus Lite baseline is operational and provides stable benchmark output.

- In this specific run, Milvus recall and QPS were nearly invariant across `nprobe`. This should be rechecked with an additional parameter sweep and index verification.

7 Re-run Checklist

- R1.** Activate environment and verify package versions.
- R2.** Ensure `sift-128-euclidean.hdf5` is present in `data/`.
- R3.** Run `python run_faiss_hdf5.py` and save console output.
- R4.** Run `python run_milvus_hdf5.py` and save console output.
- R5.** If insert fails with message-size error, reduce insert batch size.
- R6.** Archive script versions and outputs with timestamp for traceability.

8 Next Steps

- Repeat the same harness on the upcoming RX 9070 machine.
- Validate hipRAFT/hipVS and Milvus GPU path on ROCm 7.x there.
- Extend benchmark matrix to include GIST/Deep subsets.
- Add Milvus standalone server run (beyond Lite) for closer production parity.

A Teammate Run Template

This appendix provides a copy-paste template for additional benchmark runs. One filled block per run is recommended.

Run Metadata

Field	Value
Engineer	
Date/Time (UTC)	
Host name / machine ID	
GPU model / architecture (<code>gfx*</code>)	
OS / kernel	
ROCm / HIP version	
Python version	
Script commit hash / version tag	
Dataset file + checksum (if available)	

Execution Summary

- Environment setup command set used:
- Dataset acquisition method used:

- Any deviations from this runbook:
- Errors/warnings observed:

FAISS Results (copy values)

<u>Table 3: FAISS run results</u>		
Configuration	QPS	recall@10
FlatL2 (exact)		
IVF, nprobe=1		
IVF, nprobe=4		
IVF, nprobe=8		
IVF, nprobe=16		
IVF, nprobe=32		

Milvus Results (copy values)

- Insert time (s):
- Index build time (s):

<u>Table 4: Milvus IVF_FLAT run results</u>			
nprobe	QPS	p99 ms	recall@10
1			
4			
8			
16			
32			

Conclusion

- Overall status (PASS / PARTIAL / FAIL):
- Primary bottleneck:
- Recommended next action:

B Script Source: run_faiss_hdf5.py

Last validated: 2026-06-24 on host Hongkong-Ubu20 (AMD Radeon RX 6800 XT, gfx1030, Python 3.10, faiss-cpu 1.14.3).

```
import time, h5py, numpy as np, faiss
```

```
K = 10
```

```
with h5py.File("data/sift-128-euclidean.hdf5", "r") as f:
    xb = np.array(f["train"], dtype="float32")
    xq = np.array(f["test"], dtype="float32")
```

```

gt = np.array(f["neighbors"], dtype="int64")

d = xb.shape[1]
print("xb", xb.shape, "xq", xq.shape, "gt", gt.shape)

def recall_at_k(pred, gt, k):
    return np.mean([len(set(pred[i,:k]).intersection(set(gt[i,:k]))) / k
                     for i in range(pred.shape[0])])

# Exact baseline
flat = faiss.IndexFlatL2(d)
flat.add(xb)
t0 = time.time()
_, IO = flat.search(xq, K)
t1 = time.time()
print(f"[Flat] qps={xq.shape[0]/(t1-t0):.1f}, recall@{K}=1.0000")

# ANN baseline
nlist = 1024
ivf = faiss.IndexIVFFlat(faiss.IndexFlatL2(d), d, nlist, faiss.METRIC_L2)
ivf.train(xb[:200000])
ivf.add(xb)

for nprobe in [1, 4, 8, 16, 32]:
    ivf.nprobe = nprobe
    t0 = time.time()
    _, I = ivf.search(xq, K)
    t1 = time.time()
    r = recall_at_k(I, gt, K)
    print(f"[IVF] nprobe={nprobe:2d} qps={xq.shape[0]/(t1-t0):8.1f} recall@{K}={r:.4f}")

```

C Script Source: run_milvus_hdf5.py

Last validated: 2026-06-24 on host Hongkong-Ubu20 (AMD Radeon RX 6800 XT, gfx1030, Python 3.10, pymilvus 3.0.0, milvus-lite 3.0).

```

import time
import numpy as np
import h5py

from pymilvus import (
    connections, utility, FieldSchema, CollectionSchema, DataType, Collection
)

# ----- config -----
MILVUS_URI = "./milvus_sift.db"    # Milvus Lite local file
COLL = "sift1m_demo"
K = 10
NLIST = 1024
NPROBES = [1, 4, 8, 16, 32]
# -----

```

```

def recall_at_k(pred_ids, gt_ids, k):
    hits = 0
    nq = pred_ids.shape[0]
    for i in range(nq):
        hits += len(set(pred_ids[i, :k]).intersection(set(gt_ids[i, :k])))
    return hits / (nq * k)

# load hdf5
with h5py.File("data/sift-128-euclidean.hdf5", "r") as f:
    xb = np.array(f["train"], dtype=np.float32)
    xq = np.array(f["test"], dtype=np.float32)
    gt = np.array(f["neighbors"], dtype=np.int64)

d = xb.shape[1]
print(f"xb={xb.shape}, xq={xq.shape}, gt={gt.shape}, dim={d}")

# connect (Lite)
connections.connect(alias="default", uri=MILVUS_URI)

if utility.has_collection(COLL):
    utility.drop_collection(COLL)

schema = CollectionSchema(
    fields=[
        FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=False),
        FieldSchema(name="vec", dtype=DataType.FLOAT_VECTOR, dim=d),
    ],
    description="SIFT1M benchmark",
)

col = Collection(name=COLL, schema=schema)

# insert (chunked to avoid gRPC message size limit)
ids = np.arange(xb.shape[0], dtype=np.int64)
BATCH = 50000
t0 = time.time()
for s in range(0, xb.shape[0], BATCH):
    e = min(s + BATCH, xb.shape[0])
    col.insert([ids[s:e].tolist(), xb[s:e].tolist()])
col.flush()
t1 = time.time()
print(f"insert_time_s={t1 - t0:.2f}")

# build index
index_params = {
    "index_type": "IVF_FLAT",
    "metric_type": "L2",
    "params": {"nlist": NLIST},
}
t0 = time.time()

```

```

col.create_index(field_name="vec", index_params=index_params)
t1 = time.time()
print(f"index_build_time_s={t1 - t0:.2f}")

col.load()

print("\nMilvus IVF_FLAT results:")
for nprobe in NPROBES:
    search_params = {"metric_type": "L2", "params": {"nprobe": nprobe}}

    # batch search for QPS
    t0 = time.time()
    res = col.search(
        data=xq.tolist(),
        anns_field="vec",
        param=search_params,
        limit=K,
        output_fields=[],
    )
    t1 = time.time()

    total = t1 - t0
    qps = xq.shape[0] / total

    # extract ids + recall
    pred = np.array([[hit.id for hit in row] for row in res], dtype=np.int64)
    r = recall_at_k(pred, gt, K)

    # p99 latency (sampled)
    sample_n = 500
    lat_ms = []
    for i in range(sample_n):
        q = [xq[i].tolist()]
        s0 = time.time()
        _ = col.search(
            data=q,
            anns_field="vec",
            param=search_params,
            limit=K,
            output_fields=[],
        )
        s1 = time.time()
        lat_ms.append((s1 - s0) * 1000.0)
    p99 = float(np.percentile(lat_ms, 99))

    print(f"nprobe={nprobe:2d} qps={qps:8.1f} p99_ms={p99:7.2f} recall@{K}={r:.4f}")

```